Experiment 05

Aim: To automate application deployment using GitHub/Jenkins.

Theory

Continuous Integration and Continuous Deployment (CI/CD) is a software development practice in which code changes are automatically built, tested, and deployed, reducing manual effort and the chances of human error. Jenkins is one of the most widely used open-source automation servers that enables CI/CD by orchestrating these stages through configurable pipelines.

A typical Jenkins pipeline consists of the following core stages:

- Build – Compiling or preparing the application (e.g. compiling source code, installing dependencies, or packaging files).
- Test – Running automated tests to verify that the application behaves as expected.
- Deploy – Releasing the verified application to its target environment.

Jenkins pipelines can be defined in two ways: directly through a Pipeline Script written in the Jenkins UI, or through a Jenkinsfile stored in the project's source-control repository (Pipeline script from SCM). The latter approach is preferred in real-world CI/CD workflows because it version-controls the pipeline definition alongside the application code and allows Jenkins to automatically fetch the latest pipeline configuration on every run.

When integrated with GitHub, Jenkins can pull the latest code (and the Jenkinsfile) directly from a repository whenever a build is triggered, and — in a fully automated setup — a GitHub webhook can notify Jenkins to trigger a build automatically whenever new code is pushed. This experiment demonstrates both the manually-triggered local pipeline and the GitHub-integrated pipeline.

Requirements

- A GitHub account and a public repository
- Java Development Kit (JDK 21, as required by the current Jenkins LTS release)
- Jenkins (LTS release)
- A web browser
- macOS machine with Terminal access and Homebrew package manager

Procedure

1. GitHub Repository Setup

1. Created a GitHub account and a new public repository named jenkins-demo.
2. Enabled the "Add a README file" option while creating the repository.
3. Created a sample application file, index.html, containing a simple HTML page ("Hello Jenkins!").
4. Committed the file directly to the main branch of the repository using GitHub's web-based file editor.

5. Later added a Jenkinsfile (containing the Declarative Pipeline script) to the repository root, so that Jenkins could pull the pipeline definition directly from source control.

2. Installing Java and Jenkins (macOS, via Homebrew)

Jenkins and its Java dependency were installed on macOS using Homebrew, as follows:

6. Confirmed Homebrew was already installed: brew --version

```
brew --version
```

7. Installed OpenJDK 21:

```
brew install openjdk@21
```

8. Linked the JDK so system tools could locate it:

```
sudo ln -sfn /opt/homebrew/opt/openjdk@21/libexec/openjdk.jdk \
  /Library/Java/JavaVirtualMachines/openjdk-21.jdk
```

9. Added Java to the system PATH and reloaded the shell configuration:

```
echo 'export PATH="/opt/homebrew/opt/openjdk@21/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

10. Verified the installation:

```
java -version
# openjdk version "21.0.12" 2026-07-21
```

11. Installed Jenkins LTS via Homebrew:

```
brew install jenkins-lts
```

12. Started Jenkins as a background service and confirmed it was running:

```
brew services start jenkins-lts
brew services list
```

13. Retrieved the initial administrator password:

```
cat ~/.jenkins/secrets/initialAdminPassword
```

14. Opened Jenkins in the browser at http://localhost:8080, entered the administrator password, and selected "Install suggested plugins".
15. Created the first administrator account (username, password, full name, and email) to complete the setup wizard.

3. Creating and Running the Jenkins Pipeline (Local)

16. From the Jenkins Dashboard, selected New Item, entered the name GitHub-Jenkins-Demo, and chose the Pipeline project type.
17. In the Pipeline section of the configuration page, selected "Pipeline script" as the definition and entered a Declarative Pipeline:

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building the application...'
            }
```

```
        }
        stage('Test') {
            steps {
                echo 'Testing the application...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying the application...'
            }
        }
    }
}
```

18. Saved the configuration and triggered a build using Build Now.
19. Verified the result via Stage View and Console Output. All three stages (Build, Test, Deploy) completed successfully, with the console log ending in Finished: SUCCESS.

4. Integrating Jenkins with the GitHub Repository

20. Opened the pipeline's Configure page and changed the Pipeline Definition from "Pipeline script" to "Pipeline script from SCM".
21. Set the following SCM configuration:

SCM: Git

Repository URL: https://github.com/ryaancodes/jenkins-demo.git

Credentials: none (public repository)

Branch Specifier: */main

Script Path: Jenkinsfile

22. Saved the configuration and triggered a new build using Build Now.
23. Verified in the Console Output that Jenkins successfully cloned the repository and obtained the Jenkinsfile directly from GitHub before executing the pipeline stages:

```
Obtained Jenkinsfile from git https://github.com/ryaancodes/jenkins-demo.git
Cloning the remote Git repository
Cloning repository https://github.com/ryaancodes/jenkins-demo.git
Checking out Revision ...(refs/remotes/origin/main)
...
Finished: SUCCESS
```

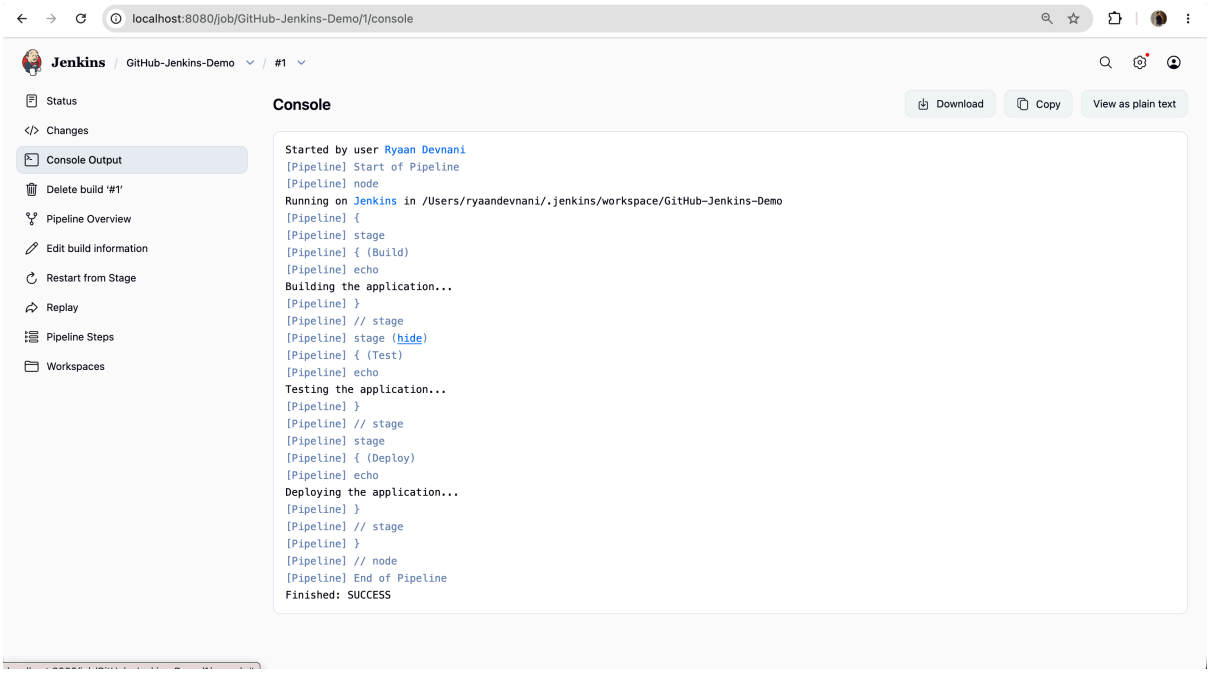
This confirmed that Jenkins was no longer relying on a locally stored script, but instead pulling the live pipeline definition from the GitHub repository — the standard practice in real CI/CD workflows.

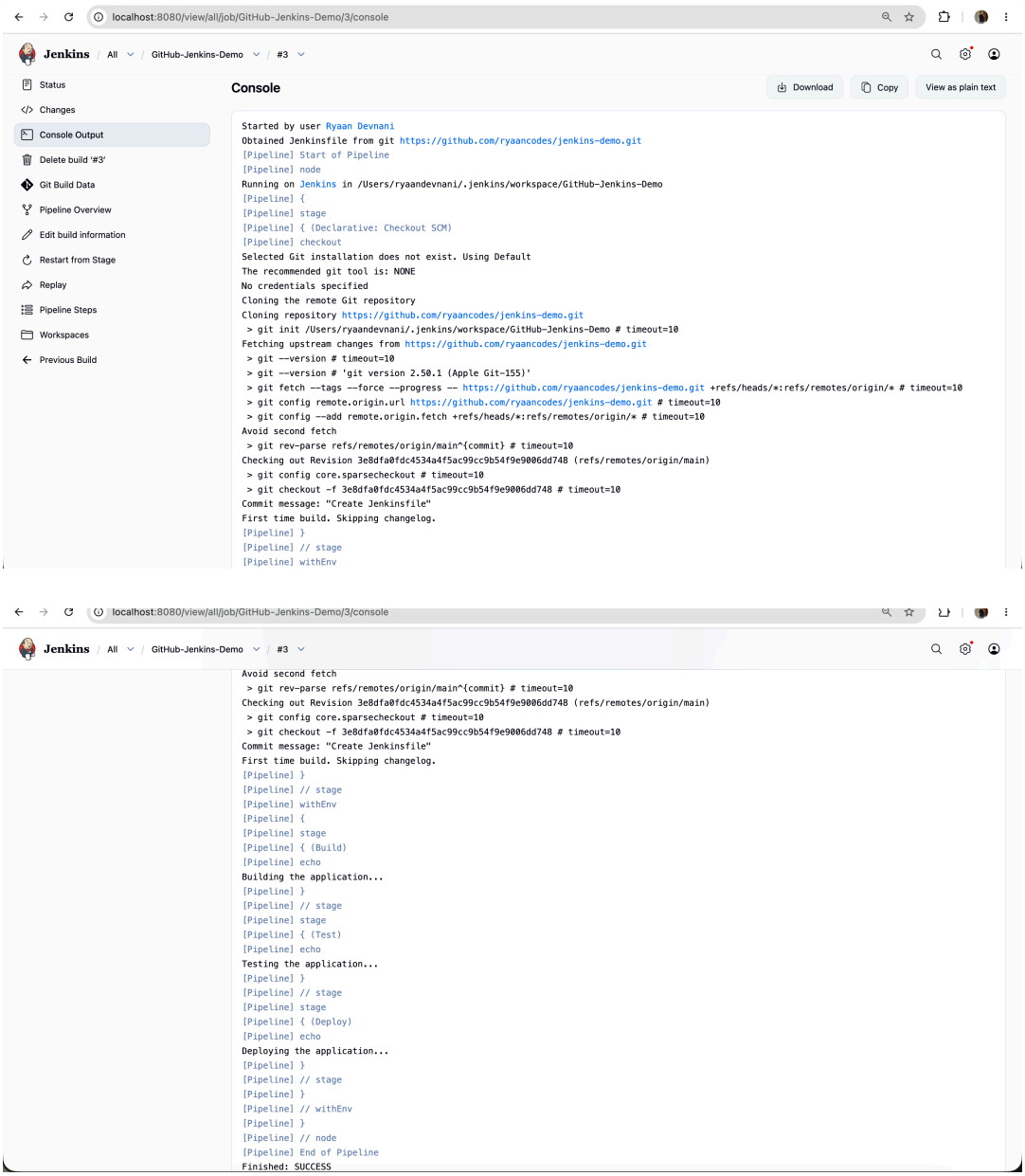
Challenges Faced and Solutions

Challenge	Solution
Re-running the Homebrew installer unnecessarily returned a curl error (HTTP 429 – rate limited).	Checked whether Homebrew was already installed using brew --version, confirmed it was, and skipped the redundant installation step entirely.

Challenge	Solution
OpenJDK 21 installed via Homebrew is keg-only and not automatically placed on the system PATH or symlinked for system Java tools to find.	Manually symlinked the JDK into /Library/Java/JavaVirtualMachines/ and appended the Homebrew OpenJDK bin directory to PATH in ~/.zshrc, then verified with java -version.
The Git branch specifier in the SCM configuration defaulted to */master, while the actual GitHub repository used main as its default branch.	Updated the Branch Specifier field to */main to match the repository's actual default branch.

Result





Conclusion

This experiment provided hands-on experience with setting up a complete CI/CD environment using Jenkins and GitHub on macOS. It covered installing Java and Jenkins via Homebrew, creating a Declarative Pipeline with distinct Build, Test, and Deploy stages, and connecting that pipeline to a live GitHub repository so that Jenkins could fetch and execute the pipeline definition directly from source control. Along the way, the experiment also demonstrated practical troubleshooting skills — verifying tool installations before reinstalling, correctly linking a keg-only Java installation, and matching Jenkins's SCM configuration to the repository's actual default branch — all of which reflect real-world challenges encountered when setting up CI/CD infrastructure. The final setup forms the foundation for a fully automated pipeline, which could be extended further by configuring a GitHub webhook so that builds are triggered automatically on every code push.